DAY OF DATA ORLANDO 2026 · SESSION 2103

Designing Cloud-Native Applications for Performance, Scalability, and Reliability

Sandeep Gundla

Saturday, October 17 · 10:15–11:15 am · Intermediate

DAY OF DATA IS MADE POSSIBLE BY OUR SPONSORS

Global Sponsor

Microsoft | AMD

Exhibitors

Seminole State College of Florida

COZYROC

SQLGrease

TRAVEL+ LEISURE

PATHFINDER

TEKsystems
Own change

In-Kind Sponsor

O'REILLY®

Community Sponsor

DevIT:>_
DevITJobs.com

Day of Data +AI
OCT 17TH 2026
Orlando, FL

THE PLAN

One hour, four live demos

Time	Segment	You will see
0:00	Intro and architecture	The two services and their stack
0:10	Database efficiency	Keyset paging, N+1 fixed
0:22	Caching	Redis cache-aside, hit ratio
0:32	Resilience	Break a dependency, watch it recover
0:47	Scale-out	Virtual threads, three replicas
0:55	Trade-offs and Q&A	A checklist to take home

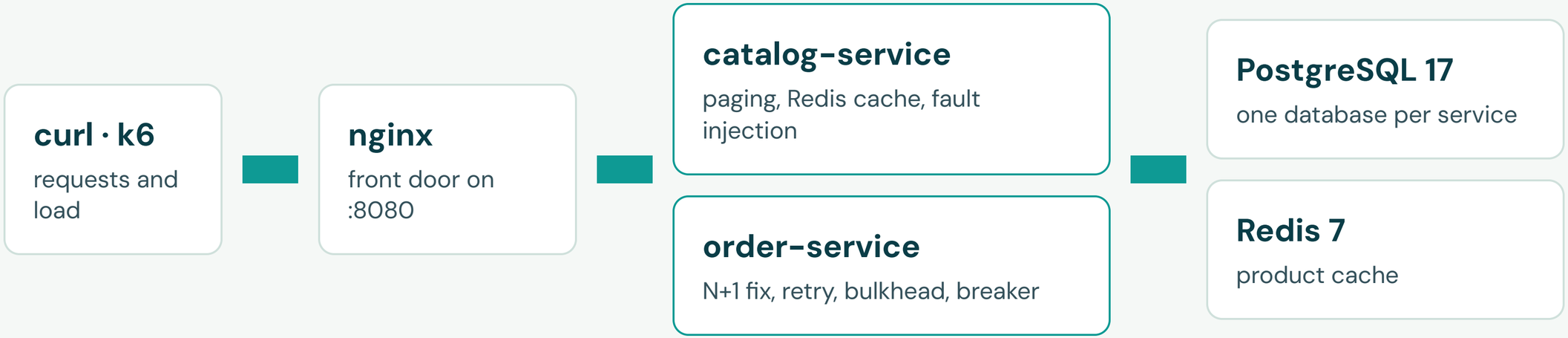
WHY THIS MATTERS

Every app will meet a slow query, a traffic spike and a failing dependency.

Three habits carry the whole hour: measure first, change one thing, measure again.

THE DEMO SYSTEM

Two services, one front door



Observability

Micrometer to Prometheus to Grafana · OpenTelemetry to Jaeger · liveness and readiness probes

FOLLOW ALONG

Run it yourself in four commands

Both systems, first command

```
git clone https://github.com/srgundla/spring-boot-perf-reliability-demo.git
```

Mac · Terminal

```
cd spring-boot-perf-reliability-demo
bash scripts/preflight.sh
docker compose up -d --build
```

Windows · PowerShell

```
cd spring-boot-perf-reliability-demo
powershell -ep bypass -File scripts\preflight.ps1
Remove-Item Alias:curl; docker compose up -d --build
```

Needs Docker Desktop (4 CPUs, 6 GB) and Git. First start takes 1–2 minutes while 500,000 rows are seeded.

PART B · DATABASE EFFICIENCY

Deep pages: OFFSET discards rows, keyset seeks

OFFSET paging

```
ORDER BY id OFFSET 495000 LIMIT 5
```

- Reads and throws away 495,000 rows
- Spring Data's Page adds a COUNT(*)
- Cost grows with depth
- Can jump to any page

Keyset paging

```
WHERE id > 495000 ORDER BY id LIMIT 5
```

- Seeks straight into the primary-key index
- No COUNT(*)
- Cost stays flat at any depth
- Only next and previous

Live: same five rows from both endpoints. Compare tookMs in the response.

PART B · DATABASE EFFICIENCY

N+1: the same 20 orders, far fewer queries

≈21

SQL statements, lazy loading

One query for the orders, then one per order for its items.

2

SQL statements, fixed

Page over ids, then one JOIN FETCH for those ids.

At 200 orders it is ≈201 against 2. Each extra statement is a round trip to a real database.

PART B · DATABASE EFFICIENCY

Three more database habits

Short transactions

Never hold a connection while calling another service. place() is not transactional; only save() is.

Select what you need

An interface projection reads four columns, not the whole entity.

Do the pool math

20 connections × 3 replicas = 60 of PostgreSQL's default 100. That is your scale-out ceiling.

LIVE DEMO · DATABASE

Paging and N+1, side by side

```
curl -s "http://localhost:8080/catalog/api/products/offset?page=99000&size=5"
curl -s "http://localhost:8080/catalog/api/products/keyset?afterId=495000&size=5"
curl -s "http://localhost:8080/orders/api/orders/slow?size=20"
curl -s "http://localhost:8080/orders/api/orders/fast?size=20"
```

tookMs

Run each twice, read the second. OFFSET is clearly slower.

sqlStatements

Same orders, about 21 statements against 2.

Windows

Run Remove-Item Alias:curl once per PowerShell window.

PART C · CACHING

Cache-aside with Redis

1 · Request
GET /api/products/42

2 · Ask Redis
Hit: answer returns, the database is never touched

3 · Miss
Query PostgreSQL (25 ms simulated expensive query)

4 · Store
Save in Redis with a 120 s TTL, then return

```
@Cacheable(cacheNames = "product", key = "#id", sync = true)
@CacheEvict(cacheNames = "product", key = "#id")
```

PART C · CACHING

Cache read-heavy, expensive, slowly changing data

Technique	What it gives you	What it costs
TTL of 120 s	Bounded staleness if an eviction is missed	Reads up to 2 minutes old
Evict on write	New price shows on the next read	Every write path must evict
sync = true	One database load per key, no stampede	Other callers wait briefly
Cache-aside	PostgreSQL stays the source of truth	App code owns the logic

Redis is another service to run, size, monitor and keep consistent. Cache on purpose, not everywhere.

LIVE DEMO · CACHING

Straight to PostgreSQL, then through Redis

```
curl -s "http://localhost:8080/catalog/api/products/42/direct"
curl -s "http://localhost:8080/catalog/api/products/42"
docker compose run --rm k6 run -e MODE=direct /scripts/cache.js
docker compose run --rm k6 run -e MODE=cached /scripts/cache.js
curl -s -X PUT "http://localhost:8080/catalog/api/products/42/price?cents=9999"
```

tookMs

About 25 ms direct, a few ms once cached.

k6 summary

Compare p(95) latency and requests per second.

Grafana

Cache hit ratio climbs when the cached run starts.

PART D · RESILIENCE

When the catalog gets slow, orders must not

Circuit breaker	Opens at 50% failures over 10 calls, then fails fast for 10 s
Bulkhead	At most 20 calls in flight to the catalog (@ConcurrencyLimit)
Retry	2 retries with backoff and jitter, only for timeouts and 5xx (@Retryable)
Timeouts	300 ms to connect, 800 ms to read: never wait forever
Fallback	Accept the order as PENDING_PRICE_CHECK and price it later

PART D · RESILIENCE

The circuit breaker has three states

CLOSED

Calls flow normally. Failures are counted over the last 10 calls.

50% or more fail

OPEN

Calls fail fast with no network use. Orders take the fallback.

After 10 seconds

HALF-OPEN

Three trial calls decide: all succeed, back to CLOSED; a failure, back to OPEN.

Grafana plots the state as a number: 0 closed, 1 open, 2 half-open.

LIVE DEMO · RESILIENCE

Break the catalog, then bring it back

```
curl -s -X POST -H "Content-Type: application/json" -d "@scripts/order.json"
"http://localhost:8080/orders/api/orders"
docker compose run --rm k6 run /scripts/orders.js
curl -s -X POST "http://localhost:8080/catalog/admin/chaos?latencyMs=1500"
curl -s "http://localhost:8080/orders/admin/breaker"
curl -s -X POST "http://localhost:8080/catalog/admin/chaos/reset"
```

1 · Slow

Order latency jumps to seconds: three attempts, all timing out.

2 · Open

Breaker OPEN, latency back to milliseconds, orders say degraded true.

3 · Recover

After reset: HALF_OPEN, then CLOSED, and orders are CONFIRMED again.

PART D · OBSERVABILITY

Metrics tell you that. Traces tell you where.

Metrics

Micrometer, Prometheus, Grafana

- Rate, errors and p95 latency
- Connection pool pressure
- Cache hit ratio
- Breaker state and fallbacks

Traces

OpenTelemetry, Jaeger

- One trace across both services
- Each retry attempt as a span
- The backoff gaps between them
- No spans at all once the breaker opens

Health probes

Spring Boot Actuator

- Liveness: restart me if I am wedged
- Readiness: send me traffic only when my dependencies are fine

PART E · SCALE-OUT

Virtual threads lift the thread limit, so limit elsewhere

What changes

- A blocked call parks a cheap virtual thread
- No pool of 200 platform threads to exhaust
- Java 25, one property to turn on

```
spring.threads.virtual.enabled=true
```

What you must add

- @ConcurrencyLimit(20) on the catalog call
- A capped connection pool: 20 per instance
- A timeout on every remote call

Without limits, cheap threads just move the pile-up to the database.

PART E · SCALE-OUT

Scale out only works if services are stateless

What makes it work

- No session state in memory
- Config from environment variables
- Readiness probe gates traffic
- Graceful shutdown drains requests

Compose today, Kubernetes tomorrow

Demo (Compose)	Cluster (k8s/)
--scale catalog=3	Deployment replicas
nginx re-resolving DNS	Service and Ingress
1 CPU per container	Requests and limits
Manual scaling	HorizontalPodAutoscaler

Scale-out moves the bottleneck. In this demo it moves to PostgreSQL.

LIVE DEMO · SCALE-OUT

One replica, then three

```
docker compose run --rm k6 run /scripts/scale.js
docker compose up -d --scale catalog=3
docker compose run --rm k6 run /scripts/scale.js
curl -si "http://localhost:8080/catalog/api/products/1"
```

k6 summary

Throughput and p(95), before against after.

X-Served-By

Repeat the curl: the replica address changes.

Grafana

Running instances shows 3; pending connections drop.

WRAP-UP

Performance, reliability and cost pull against each other

Technique	Buys you	Costs you
Redis cache	Latency, less database load	Another service, staleness
Keyset paging	Flat cost at any depth	No random page jump
Retries	Rides over short blips	Extra load on a struggling service
Breaker and fallback	Fast failure, contained blast radius	A business-approved degraded mode
More replicas	Throughput, availability	Money, more database connections
Tracing at 100%	Full detail	Overhead and storage: sample in production

TAKE IT HOME

Seven habits for the next design review

1. Measure first: p95, errors, saturation
2. Count queries per request, fix N+1 before buying hardware
3. Cache expensive, read-heavy data with a TTL and eviction
4. Timeout every remote call, retry with backoff and jitter
5. Agree the degraded behaviour with the business first
6. Stay stateless, configure by environment, probe liveness and readiness
7. Test failure on purpose, not only success

FEEDBACK

Rate this session

Scan the code with your phone. It takes a minute, and it helps the organizers and me make the next one better.

Thank you for coming.



Designing Cloud-Native Applications for Performance, Scalability, and Reliability

THANK YOU · QUESTIONS

Code, runbook and these slides

`github.com/srgundla/spring-boot-perf-reliability-demo`

Everything runs on Windows and Mac with Docker. Start with docs/SESSION-RUNBOOK.md.

Sandeep Gundla

Day of Data Orlando 2026 · Session 2103

LinkedIn `linkedin.com/in/g-sandeep-reddy`